



DETECCIÓN Y SEGUIMIENTO DE PIEZAS DE AJEDREZ MEDIANTE MATCHING Y FLUJO ÓPTICO

Sistema modular para reconstrucción de jugadas en
notación algebraica

MARTÍNEZ CAZAUX, MAURO
Ciencia e Ingeniería de Datos 2025/26

ÍNDICE

1. Introducción.....	3
1.1 Contexto del problema	3
1.2 Objetivo del trabajo	3
1.3. Alcance y restricciones	4
2. Datos de entrada y supuestos	4
2.1 Vídeo de entrada y condiciones esperadas	4
2.2 Suposiciones operativas	4
3. Detección y rectificación del tablero.....	5
3.1 Detección del tablero	5
3.2 Ordenación de esquinas	5
3.3 Homografía y tablero canónico	6
4. División en casillas y notación algebraica	6
4.1 Generación de ROIs 8x8	6
4.2 Mapeo casilla $\leftrightarrow$ notación algebraica	6
5. Detección de ocupación por casilla	7
5.1 Plantillas de vacío	7
5.2 Cálculo de ocupación y diferencias entre frames	7
6. Matching de características para identificación de pieza	7
6.1 Dataset y prototipos de piezas	7
6.2 Extracción de descriptores de características	7
6.3 Clasificación por matching de características.....	8
6.4 Estrategias de reintento y análisis múltiple de características	8
7. Inicialización del estado mediante matching	8
7.1 <i>init_state_by_matching</i> : construcción del estado inicial desde el frame inicial ...	8
7.2 Gestión de incertidumbres en la inicialización (casillas no identificadas o ambiguas)	9
8. Detección de movimiento candidato entre frames.....	9
8.1 Identificación de origen y destino por cambios de ocupación	9
8.2 Detección de capturas	9
8.3 Detección de enroque	10
9. Verificación por flujo óptico.....	10
9.1 Cálculo del flujo óptico (Farneback)	10
9.2 Validación del movimiento detectado	10
10. Integración memoria-visión y commit de jugadas	11
10.1 Integración de información visual y estado interno	11

10.2 Gestión de conflictos visión–memoria	11
10.3 Self-healing de peones	11
10.3.1 Self-healing lógico durante el commit de la jugada	12
10.3.2 Guardarraíl temprano en las primeras jugadas	12
11. Ejecución completa y salidas	12
11.1 Bucle principal de ejecución	12
11.2 Generación del vídeo anotado	12
11.3 Registro textual de jugadas.....	13
12. Limitaciones y trabajo futuro	13
12.1 Limitaciones del escenario y de los datos de entrada	13
12.2 Limitaciones del proceso de inicialización	13
12.3 Limitaciones en la lógica del juego.....	13
12.4 Trabajo futuro	14
13. Conclusiones	14
14. Bibliografía	14
15. Anexos	15
15.1 Ficheros	15
15.2 Ejecución del sistema	16
15.3 Parámetros y configuraciones	16
15.4 Ficheros de salida generados	16

1. Introducción

El análisis automático de partidas de ajedrez a partir de vídeo constituye un problema relevante dentro del ámbito de la visión por computador, al combinar retos geométricos, visuales y temporales. A diferencia de otros enfoques basados en datos estructurados, el uso de vídeo introduce incertidumbres derivadas de la perspectiva, la iluminación, el contraste entre piezas y la variabilidad entre frames consecutivos.

En este trabajo se presenta un sistema que permite detectar y reconstruir los movimientos realizados en una partida de ajedrez grabada en vídeo, combinando técnicas de detección geométrica del tablero, análisis por casillas, matching de características visuales y verificación temporal mediante flujo óptico. El objetivo final es generar, a partir del vídeo de entrada, una representación estructurada de las jugadas detectadas, junto con una visualización anotada del proceso.

El sistema se ha desarrollado y evaluado utilizando exclusivamente vídeos de partidas de ajedrez generados en la plataforma online Chess.com, en los que el tablero aparece en vista cenital y con una representación gráfica homogénea.

1.1 Contexto del problema

La detección automática de jugadas de ajedrez a partir de vídeo presenta diversas dificultades técnicas. En primer lugar, el tablero suele aparecer con distorsión de perspectiva, lo que obliga a realizar una normalización geométrica previa para poder trabajar en un sistema de coordenadas consistente.

En segundo lugar, la identificación visual de las piezas es especialmente compleja debido a la similitud entre algunas de ellas, a la variabilidad de colores y a las condiciones de iluminación, siendo los peones uno de los casos más problemáticos.

Por último, la inferencia de una jugada no puede basarse únicamente en información instantánea, sino que requiere consistencia temporal entre frames consecutivos para distinguir movimientos reales de ruido visual o detecciones espurias. Estos factores motivan el uso de un enfoque híbrido que combine información geométrica, visual y temporal.

1.2 Objetivo del trabajo

El objetivo principal de este trabajo es desarrollar un sistema capaz de detectar y reconstruir los movimientos realizados en una partida de ajedrez grabada en vídeo. Para ello, el sistema debe:

- Detectar automáticamente el tablero presente en el vídeo y normalizarlo mediante una transformación de homografía.
- Dividir el tablero normalizado en 64 casillas y establecer su correspondencia con la notación algebraica.
- Detectar cambios de ocupación entre frames consecutivos para generar candidatos de movimiento.
- Identificar la pieza implicada en cada movimiento mediante técnicas de matching de características visuales.
- Verificar la validez del movimiento detectado utilizando información de flujo óptico.

- Generar una representación textual de las jugadas detectadas, así como una visualización anotada del vídeo de entrada.

El sistema se diseña con el objetivo de ser completamente automático dentro de las restricciones del escenario considerado, priorizando la coherencia entre la información visual y el estado interno del juego.

1.3. Alcance y restricciones

El sistema desarrollado asume que el vídeo de entrada corresponde a una partida de ajedrez reproducida en la plataforma online Chess.com, donde el tablero se presenta en vista cenital, con orientación fija y sin movimientos de cámara. La representación gráfica del tablero y de las piezas es consistente a lo largo del vídeo, lo que permite aplicar técnicas de normalización geométrica y matching de características visuales de forma fiable.

El proceso de identificación de piezas se basa en técnicas de matching de características locales, apoyadas por información de ocupación por casilla y mecanismos de verificación temporal. Con el fin de mejorar la robustez del sistema frente a errores de inicialización o detección visual, se incorporan mecanismos de corrección parcial del estado interno (self-healing), especialmente orientados a la detección de peones en las primeras jugadas.

El sistema no está diseñado para operar sobre grabaciones reales con cámaras móviles, oclusiones frecuentes o variaciones bruscas de iluminación. Asimismo, no se contemplan en esta versión situaciones avanzadas del juego como promociones de peones,, detección explícita de jaque mate o análisis estratégico de la partida. El alcance del trabajo se limita a la detección y reconstrucción automática de movimientos básicos a partir de información visual extraída del vídeo.

2. Datos de entrada y supuestos

2.1 Vídeo de entrada y condiciones esperadas

El sistema toma como entrada un vídeo correspondiente a la reproducción de una partida de ajedrez en la plataforma online Chess.com. Dicho vídeo muestra el tablero completo en vista cenital, con una orientación fija y sin movimientos de cámara a lo largo de toda la secuencia.

Los vídeos utilizados presentan una resolución y tasa de fotogramas constantes, suficientes para permitir la detección estable del tablero, la extracción de regiones de interés por casilla y el análisis de cambios visuales entre frames consecutivos. La ausencia de oclusiones y la uniformidad gráfica propia de la plataforma garantizan que las piezas y las casillas mantengan una apariencia coherente durante toda la partida.

Estas características permiten aplicar de forma fiable técnicas de normalización geométrica, matching de características visuales y análisis temporal sin necesidad de realizar preprocesados complejos orientados a ruido de captura real.

2.2 Suposiciones operativas

Para el correcto funcionamiento del sistema se asumen una serie de condiciones operativas derivadas tanto del escenario considerado como de las técnicas empleadas. En primer lugar, se

presupone que el tablero permanece completamente visible durante toda la duración del vídeo y que no se producen modificaciones en su orientación ni en su escala aparente.

Asimismo, se asume que la representación visual de las piezas sigue los estilos gráficos estándar de Chess.com, lo que permite el uso de un conjunto fijo de prototipos visuales para la identificación de piezas mediante matching de características. Esta suposición es fundamental para la correcta inicialización del estado del tablero y para los mecanismos de corrección parcial del estado interno aplicados durante la ejecución.

El funcionamiento del sistema depende de manera directa de la correcta inicialización a partir del frame inicial del vídeo. En dicho frame se detecta el tablero, se aplica la normalización geométrica y se construye el estado inicial del juego mediante técnicas de matching de características visuales.

Cualquier error en la detección del tablero o en la identificación inicial de piezas en este primer frame puede propagarse a lo largo de la ejecución, afectando a la interpretación de las jugadas posteriores. Por este motivo, el sistema incorpora mecanismos de verificación temporal y corrección parcial del estado interno durante las primeras jugadas, orientados a mitigar errores de inicialización, especialmente en el caso de los peones.

Por último, se considera que entre dos frames consecutivos solo se produce, como máximo, un movimiento válido de ajedrez. Esta hipótesis permite simplificar la detección de origen y destino del movimiento y es coherente con el ritmo de reproducción de las partidas analizadas.

3. Detección y rectificación del tablero

3.1 Detección del tablero

La detección del tablero se realiza a partir del frame inicial del vídeo mediante un enfoque geométrico basado en el análisis de contornos. En primer lugar, el frame se convierte a escala de grises y se aplica un detector de bordes con el objetivo de resaltar las estructuras rectangulares presentes en la escena.

A continuación, se extraen los contornos y se selecciona aquel que presenta una forma cuadrilátera compatible con el contorno del tablero. Para ello, se emplea una aproximación poligonal que permite identificar contornos con cuatro vértices, descartando aquellos que no cumplen criterios mínimos de área y forma.

Este procedimiento resulta adecuado en el escenario considerado, dado que los vídeos procedentes de Chess.com presentan un tablero claramente delimitado, sin ruido de fondo ni elementos visuales que interfieran con su contorno.

3.2 Ordenación de esquinas

Una vez identificado el contorno cuadrilátero correspondiente al tablero, se procede a la ordenación consistente de sus cuatro vértices. Esta ordenación es necesaria para garantizar la correcta aplicación posterior de la transformación de homografía.

Las esquinas detectadas se reordenan siguiendo una convención fija (superior izquierda, superior derecha, inferior derecha e inferior izquierda), utilizando relaciones geométricas entre las coordenadas de los puntos. De este modo, se asegura que la correspondencia entre el tablero

original y su representación normalizada sea coherente independientemente del orden en el que el contorno haya sido detectado.

3.3 Homografía y tablero canónico

A partir de las esquinas ordenadas del tablero detectado, se calcula una transformación de homografía que permite mapear el tablero original a una representación canónica de tamaño fijo. Esta transformación elimina las posibles distorsiones de perspectiva presentes en el vídeo de entrada y permite trabajar en un sistema de coordenadas normalizado.

El tablero canónico obtenido presenta una orientación fija y una escala uniforme, lo que facilita la división posterior en casillas y el análisis visual por regiones de interés. Todas las operaciones posteriores del sistema se realizan sobre esta representación normalizada del tablero, garantizando consistencia geométrica a lo largo de toda la ejecución.

4. División en casillas y notación algebraica

4.1 Generación de ROIs 8x8

Una vez obtenido el tablero en su representación canónica, se procede a su división en una cuadrícula regular de 8×8 casillas. Esta división se realiza de forma uniforme, utilizando el tamaño fijo del tablero normalizado para calcular los límites de cada región de interés (ROI) correspondiente a una casilla.

Cada ROI representa una casilla individual del tablero y constituye la unidad básica de análisis para las etapas posteriores del sistema. Sobre estas regiones se aplican los métodos de detección de ocupación, matching de características y validación temporal, evitando interferencias entre casillas adyacentes.

El uso de una cuadrícula regular sobre el tablero canónico garantiza que las coordenadas de las ROIs sean consistentes a lo largo de toda la ejecución, independientemente del frame analizado.

4.2 Mapeo casilla $\leftrightarrow$ notación algebraica

A cada casilla de la cuadrícula 8×8 se le asigna una correspondencia directa con la notación algebraica estándar del ajedrez. Para ello, se define un sistema de coordenadas fila–columna sobre el tablero canónico, que posteriormente se transforma en coordenadas algebraicas del tipo letra–número.

Esta correspondencia permite traducir cualquier movimiento detectado en términos de índices internos del sistema a una representación comprensible y estándar. El mapeo se mantiene constante durante toda la ejecución y es utilizado tanto para la generación de la salida textual de jugadas como para la anotación visual del vídeo.

5. Detección de ocupación por casilla

5.1 Plantillas de vacío

La detección de ocupación de las casillas se basa en la comparación visual entre cada región de interés y una referencia de casilla vacía. Para ello, se construyen plantillas representativas de casillas sin pieza, diferenciando entre casillas claras y oscuras del tablero.

Estas plantillas se obtienen a partir del tablero canónico y se utilizan como referencia para medir el grado de similitud entre una ROI y el aspecto esperado de una casilla vacía. Este enfoque permite identificar de forma eficiente si una casilla contiene o no una pieza, independientemente del tipo de pieza presente.

5.2 Cálculo de ocupación y diferencias entre frames

Para cada frame analizado, el sistema evalúa la ocupación de las 64 casillas comparando sus ROIs con las plantillas de vacío correspondientes. A partir de esta evaluación se genera una máscara de ocupación binaria que indica, para cada casilla, si se considera ocupada o vacía.

La detección de movimientos se apoya en la comparación de las máscaras de ocupación entre frames consecutivos. Los cambios en el estado de ocupación permiten identificar casillas candidatas de origen y destino del movimiento, así como detectar situaciones de captura cuando una casilla previamente ocupada pasa a considerarse vacía sin que se detecte una nueva ocupación alternativa.

Este análisis por diferencias de ocupación constituye el primer filtro para la detección de jugadas, reduciendo el espacio de búsqueda y facilitando la integración posterior con técnicas de matching de características y verificación temporal.

6. Matching de características para identificación de pieza

6.1 Dataset y prototipos de piezas

Para la identificación de las piezas implicadas en cada movimiento, el sistema utiliza un conjunto de prototipos visuales correspondientes a las distintas piezas de ajedrez. Dichos prototipos se extraen de la representación gráfica estándar de Chess.com y se organizan en un dataset por tipo de pieza.

Este conjunto de referencias se emplea como base para el proceso de matching de características, permitiendo comparar regiones de interés del tablero con ejemplos visuales conocidos. El uso de prototipos específicos del entorno considerado garantiza coherencia visual entre las piezas detectadas en el vídeo y las utilizadas como referencia.

6.2 Extracción de descriptores de características

Sobre cada región de interés candidata se extraen descriptores locales utilizando el algoritmo AKAZE. Este método permite obtener características invariantes a pequeñas variaciones de escala y rotación, resultando adecuado para la identificación visual de piezas en el tablero normalizado.

La extracción de descriptores se aplica tanto a las ROIs del tablero como a los prototipos del dataset, generando representaciones comparables que sirven como entrada para el proceso de

matching. En aquellas casillas donde no se detectan suficientes características visuales, el sistema marca la identificación como incierta, delegando la decisión final a mecanismos posteriores de validación.

6.3 Clasificación por matching de características

La identificación del tipo de pieza se realiza mediante un proceso de matching entre los descriptores extraídos de la ROI analizada y los correspondientes a los prototipos del dataset. Para ello, se emplea un comparador de fuerza bruta junto con criterios de decisión basados en la calidad y consistencia de las correspondencias obtenidas.

El resultado del matching proporciona una etiqueta visual candidata para la pieza presente en la casilla analizada, junto con una medida de confianza asociada. Esta información se utiliza de forma integrada con la detección de ocupación y el estado interno del tablero, evitando decisiones basadas exclusivamente en una única fuente visual.

6.4 Estrategias de reintento y análisis múltiple de características

En aquellos casos en los que el proceso de matching de características no produce una identificación fiable de la pieza presente en una casilla, el sistema no toma una decisión inmediata basada en un único análisis visual. En su lugar, se aplican estrategias de reintento que consisten en realizar múltiples análisis de características sobre la misma región de interés.

Estos análisis adicionales pueden implicar variaciones en los parámetros de extracción de descriptores, el uso de diferentes subconjuntos de características detectadas o la repetición del matching en distintos frames consecutivos. El objetivo de este enfoque es reducir la probabilidad de falsos negativos derivados de condiciones visuales adversas, como bajo contraste, escasez de puntos clave o ruido residual en la imagen.

La decisión final sobre el tipo de pieza se adopta únicamente cuando se alcanza un nivel mínimo de consistencia entre los resultados obtenidos, o bien cuando la información visual se combina con otros indicadores del sistema, como la detección de ocupación o el estado interno del tablero. Este mecanismo permite mejorar la robustez del sistema sin comprometer la coherencia temporal de las jugadas detectadas.

7. Inicialización del estado mediante matching

7.1 *init_state_by_matching*: construcción del estado inicial desde el frame inicial

El sistema construye un estado interno del tablero a partir del **frame inicial** del vídeo, ya que este constituye la referencia a partir de la cual se interpretan las jugadas posteriores. Tras detectar y normalizar el tablero mediante homografía, se divide el tablero canónico en 64 casillas y se analiza cada casilla para determinar su contenido inicial.

La inicialización se realiza combinando dos fuentes de información: (1) la **detección de ocupación por casilla** (casilla vacía u ocupada) y (2) la **identificación visual de la pieza mediante matching de características**. En las casillas consideradas ocupadas, el sistema ejecuta el clasificador por matching (AKAZE + matching con prototipos) para asignar una etiqueta de pieza y construir así el estado inicial del tablero.

El estado inicial generado se almacena como una estructura interna que asigna a cada casilla una etiqueta de contenido (vacía o pieza identificada). A partir de este momento, dicho estado se utiliza como “memoria” del tablero y se actualiza progresivamente conforme se detectan movimientos en los frames siguientes.

7.2 Gestión de incertidumbres en la inicialización (casillas no identificadas o ambiguas)

La inicialización visual puede presentar incertidumbre en determinadas casillas, por ejemplo cuando la región de interés contiene pocos puntos clave, existe bajo contraste o el resultado del matching no es suficientemente consistente. En estos casos, el sistema **no fuerza una identificación** si no dispone de evidencia suficiente, dejando la casilla marcada como no concluyente para evitar introducir errores sistemáticos en el estado inicial.

Dado que el estado interno se arrastra a lo largo de la ejecución, un error en el frame inicial puede propagarse e impactar en la interpretación de movimientos posteriores. Por este motivo, el sistema incorpora mecanismos de robustez durante la ejecución: (i) verificación temporal de los movimientos detectados y (ii) corrección parcial del estado cuando existe evidencia fuerte de ocupación o movimiento que contradice la memoria. En particular, se contemplan mecanismos orientados a mitigar fallos de inicialización en peones, de forma que una pieza no detectada en el estado inicial pueda incorporarse de manera controlada cuando se mueve por primera vez (self-healing).

En conjunto, este diseño permite que la inicialización sea lo más fiel posible al frame inicial sin comprometer la coherencia del sistema ante casos puntuales de incertidumbre visual.

8. Detección de movimiento candidato entre frames

8.1 Identificación de origen y destino por cambios de ocupación

La detección de movimientos entre frames consecutivos se basa inicialmente en el análisis de cambios en la ocupación de las casillas del tablero. A partir de las máscaras de ocupación calculadas para dos frames consecutivos, el sistema identifica aquellas casillas cuyo estado ha cambiado, generando un conjunto reducido de candidatas a origen y destino del movimiento.

En el caso más habitual, un movimiento simple se caracteriza por la transición de una casilla previamente ocupada a vacía (casilla origen) y de una casilla previamente vacía a ocupada (casilla destino). Esta información permite proponer un movimiento candidato sin necesidad de analizar exhaustivamente todas las casillas del tablero.

El uso de diferencias de ocupación como primer filtro reduce significativamente el coste computacional del sistema y limita el análisis posterior a un número muy reducido de casillas relevantes.

8.2 Detección de capturas

El sistema contempla explícitamente situaciones de captura, en las que una pieza se desplaza a una casilla previamente ocupada por una pieza rival. En estos casos, el análisis de ocupación puede no mostrar una nueva casilla ocupada, ya que el destino ya contenía una pieza en el frame anterior.

Para manejar esta situación, el sistema identifica escenarios en los que una casilla ocupada pasa a vacía sin que se detecte una nueva ocupación alternativa clara, interpretando este patrón como una posible captura. La casilla de destino se infiere entonces a partir del análisis conjunto de los cambios de ocupación y del estado interno del tablero.

Este tratamiento específico permite detectar capturas sin necesidad de analizar explícitamente el tipo de pieza capturada en el momento del movimiento.

8.3 Detección de enroque

El sistema incorpora un tratamiento específico para la detección del enroque, dado que este movimiento implica el desplazamiento simultáneo de dos piezas. La detección se basa en patrones característicos de cambio de ocupación en las casillas implicadas, compatibles con un movimiento de rey y torre en el mismo turno.

Cuando se detecta un patrón de ocupación coherente con un enroque, el sistema genera directamente el movimiento correspondiente, diferenciando entre enroque corto y largo según las casillas afectadas. Este caso se maneja de forma independiente al flujo estándar de detección de movimientos simples, garantizando una interpretación correcta del evento.

9. Verificación por flujo óptico

9.1 Cálculo del flujo óptico (Farnebäck)

Para validar los movimientos candidatos detectados mediante cambios de ocupación, el sistema utiliza información de flujo óptico entre frames consecutivos. En concreto, se calcula el flujo óptico denso utilizando el método de Farnebäck sobre la representación canónica del tablero, lo que permite estimar el desplazamiento aparente de los píxeles entre dos frames.

El cálculo se realiza sobre el tablero normalizado para garantizar coherencia espacial y evitar distorsiones debidas a la perspectiva original del vídeo. A partir del campo de flujo obtenido, se extrae una medida de magnitud que resume la intensidad del movimiento detectado en distintas regiones del tablero.

9.2 Validación del movimiento detectado

La validación del movimiento se basa en comprobar que la información de flujo óptico es consistente con el movimiento candidato propuesto a partir de los cambios de ocupación. En particular, el sistema verifica que las regiones correspondientes a las casillas origen y destino presentan una magnitud de flujo significativa, compatible con el desplazamiento de una pieza entre ambas posiciones.

Si el patrón de flujo observado respalda el movimiento candidato, este se considera válido y se pasa a las etapas posteriores de integración con el estado interno del tablero. En caso contrario, el movimiento se descarta o se mantiene en espera hasta que exista confirmación suficiente en frames posteriores.

Este proceso de verificación actúa como un filtro temporal que permite reducir falsos positivos derivados de ruido visual, pequeñas variaciones de iluminación o errores puntuales en la detección

de ocupación, reforzando la coherencia global del sistema sin imponer una dependencia exclusiva del flujo óptico.

10. Integración memoria-visión y commit de jugadas

10.1 Integración de información visual y estado interno

Una vez validado un movimiento candidato mediante cambios de ocupación y verificación por flujo óptico, el sistema procede a integrarlo con el estado interno del tablero. Este estado actúa como una memoria persistente que refleja la configuración actual del juego y se actualiza de forma incremental tras cada jugada confirmada.

La integración combina tres fuentes de información: (i) el estado previo del tablero, (ii) la identificación visual de la pieza implicada mediante matching de características y (iii) la información geométrica del movimiento detectado (origen y destino). Esta combinación permite mantener coherencia entre la percepción visual y la representación interna del juego, evitando decisiones basadas exclusivamente en una única fuente.

10.2 Gestión de conflictos visión–memoria

Durante la ejecución pueden aparecer discrepancias entre la información visual extraída del frame actual y el estado interno almacenado en memoria. Estas situaciones pueden deberse a errores de inicialización, fallos puntuales del matching o condiciones visuales adversas.

Cuando se detecta un conflicto entre la etiqueta visual obtenida y el contenido esperado según la memoria, el sistema no actualiza el estado de forma inmediata. En su lugar, se aplican criterios de decisión que priorizan la consistencia temporal y la evidencia acumulada, permitiendo posponer o descartar la actualización hasta disponer de información suficiente.

Este mecanismo evita la propagación de errores aislados y contribuye a mantener la estabilidad del estado interno a lo largo de la ejecución.

10.3 Self-healing de peones

El mecanismo de self-healing se aplica de forma específica a los peones debido a las dificultades particulares que presenta su detección visual. A diferencia de otras piezas, los peones ocupan una menor superficie, generan menos puntos clave y presentan una apariencia visual más homogénea, lo que incrementa la probabilidad de fallos en el matching de características, especialmente en condiciones de bajo contraste o iluminación desfavorable.

Dado que los peones son además las piezas que más frecuentemente se mueven durante una partida, un error en su detección inicial puede propagarse rápidamente y afectar de forma significativa a la coherencia del estado interno. Por este motivo, el sistema incorpora mecanismos específicos de corrección parcial orientados exclusivamente a peones, evitando extender este tipo de corrección a otras piezas cuyo patrón visual resulta más estable y distintivo.

10.3.1 Self-healing lógico durante el commit de la jugada

En determinados casos, el estado interno puede indicar que una casilla está vacía mientras que la información visual confirma la presencia de una pieza y la detección de ocupación respalda dicha observación. Cuando esta situación se produce y el matching visual identifica la pieza como un peón, el sistema aplica un mecanismo de corrección parcial del estado interno, insertando el peón en la casilla correspondiente antes de realizar la actualización del movimiento.

Este proceso, denominado self-healing, permite corregir errores de inicialización o detección sin introducir cambios arbitrarios en el estado del tablero, limitando la corrección a casos en los que existe evidencia visual suficiente

10.3.2 Guardarraíl temprano en las primeras jugadas

Adicionalmente, durante las primeras jugadas de la partida, el sistema aplica un guardarraíl específico orientado a mitigar errores frecuentes en la detección inicial de peones. Si en estas fases tempranas la identificación visual resulta inconclusa pero la detección de ocupación y el movimiento observado son consistentes, el sistema fuerza de manera controlada la aceptación de la pieza como peón.

Este guardarraíl introduce una heurística basada en el comportamiento típico de las partidas de ajedrez, utilizada únicamente como mecanismo de recuperación temprana.

Este mecanismo se aplica únicamente en un número limitado de jugadas iniciales y tiene como objetivo facilitar la recuperación de peones que no hayan sido correctamente identificados en el frame inicial, evitando que estos errores se propaguen al resto de la partida.

11. Ejecución completa y salidas

11.1 Bucle principal de ejecución

Una vez inicializado el estado del tablero a partir del frame inicial, el sistema procesa el vídeo de entrada de forma secuencial frame a frame. Para cada nuevo frame, se aplica la normalización geométrica previamente calculada y se extrae el tablero canónico, sobre el cual se realizan las etapas de detección de ocupación, generación de candidatos de movimiento y verificación por flujo óptico.

Los movimientos candidatos que superan los criterios de validación se integran con el estado interno del tablero mediante la lógica de commit de jugadas, actualizando la memoria del sistema y preparando el estado para el procesamiento del frame siguiente. Este procedimiento iterativo garantiza la coherencia temporal de las jugadas detectadas a lo largo de toda la secuencia.

11.2 Generación del vídeo anotado

Como salida visual del sistema, se genera un vídeo anotado que muestra el tablero normalizado junto con información textual relativa a las jugadas detectadas. Las anotaciones se actualizan dinámicamente conforme se confirman nuevos movimientos, permitiendo visualizar de manera directa el resultado del análisis realizado por el sistema.

El vídeo anotado constituye una herramienta de validación cualitativa del funcionamiento del sistema, facilitando la inspección visual de los movimientos detectados y su correspondencia con la evolución real de la partida.

11.3 Registro textual de jugadas

De forma complementaria, el sistema produce un registro textual de las jugadas detectadas, en el que se almacena de manera secuencial la información correspondiente a cada movimiento confirmado. Este registro incluye la notación algebraica asociada a cada jugada y se genera de forma sincronizada con el procesamiento del vídeo.

El fichero de salida permite analizar posteriormente la secuencia de movimientos sin necesidad de reproducir el vídeo, y sirve como representación estructurada del resultado final del sistema.

12. Limitaciones y trabajo futuro

12.1 Limitaciones del escenario y de los datos de entrada

El sistema desarrollado está específicamente diseñado para operar sobre vídeos de partidas de ajedrez reproducidas en la plataforma Chess.com, en los que el tablero aparece en vista cenital, con orientación fija y sin oclusiones. Esta restricción permite simplificar el tratamiento geométrico y visual del problema, pero limita la aplicabilidad del sistema a entornos gráficos con características similares.

No se contempla el uso de grabaciones reales con cámaras móviles, variaciones significativas de perspectiva, cambios bruscos de iluminación u oclusiones parciales del tablero, ya que dichos escenarios requerirían técnicas adicionales de detección y seguimiento que quedan fuera del alcance de este trabajo.

12.2 Limitaciones del proceso de inicialización

El funcionamiento del sistema depende de manera directa de la correcta inicialización del estado del tablero a partir del frame inicial del vídeo. Aunque se emplean técnicas de matching de características para identificar las piezas presentes en dicho frame, errores persistentes en esta etapa pueden propagarse a lo largo de la ejecución.

Si bien el sistema incorpora mecanismos de verificación temporal y corrección parcial del estado interno para mitigar estos efectos, dichas estrategias no garantizan la recuperación completa ante fallos graves o sistemáticos en la detección inicial de piezas.

12.3 Limitaciones en la lógica del juego

En términos de reglas ajedrecísticas, el sistema se centra en la detección de movimientos básicos y en el tratamiento de casos específicos como capturas y enroques. No se abordan situaciones más avanzadas como promociones de peones, ambigüedades complejas de notación SAN, detección explícita de jaque mate o análisis estratégico de la partida.

Estas limitaciones responden a una decisión de diseño orientada a mantener la coherencia del sistema y a centrar el trabajo en los aspectos visuales y temporales del problema.

12.4 Trabajo futuro

Como líneas de trabajo futuro, se plantea la extensión del sistema a otros entornos gráficos o plataformas distintas de Chess.com, así como la adaptación del enfoque a escenarios de captura real mediante cámara.

Asimismo, podrían incorporarse mecanismos más avanzados de validación y resolución de ambigüedades, ampliar el tratamiento de reglas especiales del ajedrez y realizar una evaluación cuantitativa del rendimiento del sistema sobre conjuntos de datos más amplios y variados.

13. Conclusiones

En este trabajo se ha desarrollado un sistema automático para la detección y reconstrucción de movimientos en partidas de ajedrez a partir de vídeo, abordando el problema desde una perspectiva de visión por computador clásica. El enfoque propuesto combina detección geométrica del tablero, análisis por casillas, matching de características visuales y verificación temporal mediante flujo óptico, integrando estas fuentes de información en un estado interno persistente del juego.

El sistema parte de la detección y normalización geométrica del tablero en el frame inicial, a partir de la cual se construye una representación interna del estado del juego mediante técnicas de matching de características. Sobre esta base, los movimientos se infieren a partir de cambios de ocupación entre frames consecutivos y se validan mediante información de flujo óptico, lo que permite reducir errores derivados de ruido visual o detecciones espurias.

Uno de los aspectos más relevantes del diseño es la integración de mecanismos de corrección parcial del estado interno, orientados a mitigar errores de inicialización y detección visual, especialmente en el caso de los peones. Estos mecanismos no garantizan una recuperación completa ante cualquier fallo, pero contribuyen a mantener la coherencia del sistema en situaciones frecuentes dentro del escenario considerado.

El trabajo demuestra que, en un entorno gráfico controlado como el proporcionado por la plataforma Chess.com, es posible reconstruir de forma consistente la secuencia de jugadas de una partida utilizando técnicas clásicas de visión por computador y un diseño cuidadoso del flujo de información. El sistema desarrollado constituye una base sólida y extensible para futuros trabajos orientados a escenarios más complejos, mayor robustez visual o una cobertura más amplia de las reglas del juego.

14. Bibliografía

- Bay, H., Tuytelaars, T., & Van Gool, L. (2006). **SURF: Speeded up robust features**. *European Conference on Computer Vision (ECCV)*, 404–417. https://doi.org/10.1007/11744023_32
- Bradski, G. (2000). **The OpenCV Library**. *Dr. Dobb's Journal of Software Tools*.
- Farnebäck, G. (2003). **Two-frame motion estimation based on polynomial expansion**. *Proceedings of the 13th Scandinavian Conference on Image Analysis*, 363–370. https://doi.org/10.1007/3-540-45103-X_50

Lowe, D. G. (2004). **Distinctive image features from scale-invariant keypoints**. *International Journal of Computer Vision*, 60(2), 91–110. <https://doi.org/10.1023/B:VISI.0000029664.99615.94>

OpenCV Team. (2023). **OpenCV documentation**. <https://docs.opencv.org>

Rublee, E., Rabaud, V., Konolige, K., & Bradski, G. (2011). **ORB: An efficient alternative to SIFT or SURF**. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2564–2571. <https://doi.org/10.1109/ICCV.2011.6126544>

Schmid, C., Mohr, R., & Bauckhage, C. (2000). **Evaluation of interest point detectors**. *International Journal of Computer Vision*, 37(2), 151–172. <https://doi.org/10.1023/A:1008199403446>

Chess.com. (2024). **Chess.com online chess platform**. <https://www.chess.com>

15. Anexos

15.1 Ficheros

El proyecto se organiza en una estructura de directorios sencilla que separa los datos utilizados por el sistema, el código fuente y los ficheros de prueba. A continuación se describe la estructura general del directorio entregado:

main.py

dataset/

├─ B/

├─ EMPTY/

├─ K/

├─ N/

├─ P/

├─ Q/

├─ R/

test/

├─ partida.mp4

src/

├─ chess_logic.py

├─ classification.py

├─ config.py

├─ features.py

├─ geometry.py

├─ grid.py

- |— motion.py
- |— occupancy.py
- |— video_processing.py

El directorio `dataset/` contiene el conjunto de imágenes de referencia utilizadas como prototipos visuales para la identificación de piezas mediante matching de características. Cada subdirectorio corresponde a un tipo de pieza, siguiendo la notación empleada en el sistema (K: Rey, Q: Dama, R: Torre, B: Alfil, N: Caballo, P: Peón, EMPTY: casilla vacía).

El directorio `test/` incluye los vídeos de entrada utilizados para la evaluación del sistema. En particular, se emplea un vídeo de una partida de ajedrez reproducida en la plataforma Chess.com, que constituye el caso de estudio principal del trabajo.

El directorio `src/` agrupa el código fuente del sistema, organizado en módulos independientes que implementan las distintas etapas del pipeline descrito en esta memoria. Esta organización modular permite una mayor claridad, mantenibilidad y trazabilidad entre el código y las secciones teóricas del documento.

15.2 Ejecución del sistema

La ejecución del sistema se realiza a través del fichero principal `main.py`, que recibe como parámetros las rutas de los ficheros de entrada y salida necesarios para el procesamiento del vídeo.

Durante la ejecución, el programa procesa el vídeo frame a frame, detecta y valida las jugadas realizadas y genera como salida un vídeo anotado con la información visual de los movimientos detectados, así como un fichero de texto que recoge la secuencia de jugadas en notación algebraica.

15.3 Parámetros y configuraciones

Los parámetros y umbrales que controlan el comportamiento del sistema (tales como tamaños del tablero canónico, criterios de ocupación, umbrales de matching, validación por flujo óptico y mecanismos de corrección parcial del estado interno) se encuentran centralizados en el fichero `config.py`.

Estos valores han sido ajustados de forma empírica en función del escenario considerado (vídeos de partidas reproducidas en Chess.com) y se documentan directamente en el código. No se ha llevado a cabo un proceso sistemático de optimización de parámetros, ya que el objetivo principal del trabajo es el diseño y validación funcional del pipeline propuesto.

15.4 Ficheros de salida generados

Como resultado de la ejecución del sistema, se generan dos tipos principales de ficheros de salida: un vídeo anotado que muestra visualmente las jugadas detectadas sobre el tablero, y un fichero de texto que contiene la secuencia de movimientos confirmados en notación algebraica.

Estos ficheros permiten realizar una validación cualitativa del funcionamiento del sistema y analizar los resultados obtenidos sin necesidad de reejecutar el procesamiento completo del vídeo.

